LAPORAN PRAKTIKUM 6 PEMROGRAMAN BERIORIENTASI OBJEK

ABSTRACTION



disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : Afdhal Dinilhak, M.Kom,

Asisten Praktikum : Siti Fadhilah Rahmi

Tanggal Praktikum : 5 November 2026

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

A. LANGKAH LANGKAH PRAKTIKUM

L1 Abstract class dan abstract method

L1.a-Product

```
package shop.model;

 public abstract class Product {
    private final String sku;
    private final String name;
    private final long price;

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be
blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be
blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price cannot be
negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }
    public String getSku() {
        return sku;
    }

    public String getName() {
        return name;
    }

    public long getPrice() {
        return price;
    }
    public long getFinalPrice() {
        return price;
    }
    public abstract String category();
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" +
getFinalPrice();
```

```
    }
}
```

Penjelasan: Kelas Product dirancang sebagai abstract class karena mewakili konsep umum barang di toko yang tidak bisa diinstansiasi secara langsung. Kelas ini menyimpan data seperti sku, name, dan price dengan validasi nilai pada konstruktor protected. Terdapat abstract method category() yang mewajibkan setiap kelas turunan menentukan kategorinya sendiri, serta method getInfo() yang memanfaatkan polimorfisme untuk memanggil category() dan getFinalPrice().

L1.b-Class PhysicalProduct

```
package shop.model;

public class PhysicalProduct extends Product implements
shop.shipping.Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int
weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be
greater than zero");
        }
        this.weightGrams = weightGrams;
    }
    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Penjelasan: Kelas PhysicalProduct merupakan kelas turunan dari Product yang merepresentasikan produk fisik. Kelas ini menambahkan atribut

weightGrams (dengan validasi > 0) dan mengimplementasikan method category() yang mengembalikan string "Fisik"

L1.c-Class DigitalProduct

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable
{
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT = 10;

    public DigitalProduct(String sku, String name, long price,
String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must
not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }
    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }

    @Override
    public String category() {
        return "Digital";
    }
}
```

Penjelasan: Kelas DigitalProduct adalah kelas turunan Product untuk produk digital. Kelas ini menambahkan atribut downloadUrl dengan validasi tidak boleh kosong dan mengimplementasikan method category() yang mengembalikan string "Digital"

L1.d-Class ShippingService

```
package shop.shipping;

public abstract class ShippingService {
    private final String name;
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be
blank");
        }
        this.name = name;
    }
    public String getName() {
        return name;
    }
    protected abstract long ratePerKg();
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
    }
    long kilograms = (weightGrams + 999L)/1000;
    return kilograms * ratePerKg();
    }
}
```

Penjelasan: Kelas ShippingService dirancang sebagai abstract class untuk layanan pengiriman. Metode perhitungan ongkir calculateCost(int weightGrams) dideklarasikan di sini dengan aturan pembulatan ke atas. Kelas ini memiliki abstract method ratePerKg() yang harus diimplementasikan oleh masing-masing penyedia kurir untuk menentukan tarif per kilogramnya.

L1.e-Class RegularShipping dan ExpressShipping.

```
package shop.shipping;
```

```java
public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Reguler");
    }
    @Override
    protected long ratePerKg() {
        return 10_000;
    }

}
```

Penjelasan: Kelas RegularShipping adalah kelas konkret turunan dari ShippingService yang merepresentasikan opsi pengiriman reguler. Kelas ini memanggil super("Reguler") pada konstruktornya dan meng-override method ratePerKg() dengan mengembalikan nilai 10000 per kilogram.

```java
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18000;
    }
}
```

Penjelasan: Kelas ExpressShipping adalah kelas konkret turunan dari ShippingService. Kelas ini memanggil super("Ekspres") pada konstruktornya dan meng-override method ratePerKg() dengan mengembalikan nilai 18000 per kilogram.

Checkpoint L1: SelfCheck L1.1–L1.7 lulus.

```
-- L1 - Abstract class & abstract method
  [LULUS] L1.1 Product adalah abstract class
  [LULUS] L1.2 Product punya abstract method category()
  [LULUS] L1.3 PhysicalProduct & DigitalProduct adalah subclass konkret Product
  [LULUS] L1.4 Konstruktor menolak harga negatif (IllegalArgumentException)
  [LULUS] L1.5 getInfo() PhysicalProduct sesuai format
  [LULUS] L1.6 ShippingService abstract dengan abstract method ratePerKg()
  [LULUS] L1.7 calculateCost(): Reguler 700g=10000, 1001g=20000, 0g=0; Ekspres 700g=18000
```

L2 Interface dan program to an interface

L2.a-Shippable

```java
package shop.shipping;

public interface Shippable {
    int getWeightGrams();
}
```

Penjelasan: Interface Shippable mendefinisikan capability untuk objek yang memiliki berat dan dapat dikirim via kurir melalui kontrak method getWeightGrams(). PhysicalProduct mengimplementasikan interface ini, sedangkan DigitalProduct tidak karena produk digital tidak memerlukan pengiriman fisik.

L2.b-Interface PaymentMethod

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);
}
```

Penjelasan: Interface PaymentMethod bertindak sebagai kontrak bagi seluruh metode pembayaran di toko online dengan menyediakannya method getName() dan pay(long amount).

L2.c-Class EWalletPayment

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;
```

```
    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be
blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be
negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Penjelasan: Kelas EWalletPayment mengimplementasikan PaymentMethod untuk pembayaran via dompet digital. Konstruktor mengunci nilai provider dan saldo dasar. Method pay(amount) akan memeriksa validasi saldo dan mengurangi saldo jika transaksi berhasil.

L2.d-Class BankTransferPayment

```
package shop.payment;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Bank name must not
be blank");
        }
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
}
```

Penjelasan: Kelas BankTransferPayment mengimplementasikan PaymentMethod. Method pay(amount) mengembalikan nilai true selama nominal nominal pembayaran > 0.

L2.e-Class CheckoutService

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {
    public long subtotal(List<Product> items) {
```

```
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }
    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }
    public long total(List<Product> items, ShippingService shipping)
{
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }
    public boolean checkout(List<Product> items, ShippingService
shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        long totalAmount = total(items, shipping);
        return payment.pay(totalAmount);
    }
}
```

Penjelasan: CheckoutService menjaga alur transaksi. Method totalWeight() melakukan pengecekan instanceof Shippable untuk menghitung total berat barang fisik saja. Method total() dan checkout() berinteraksi hanya dengan abstraksi ShippingService dan PaymentMethod tanpa terikat pada kelas tertentu.

L2.f-ShopApp

```
package shop.app;

import java.util.List;

import shop.model.Address;
```

```java
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
       Product coffee = new PhysicalProduct("KOPI-001", "Kopi
Arabika Gayo 250g", 85_000, 300);
       Product rendang = new PhysicalProduct("RDG-002", "Rendang
Kemasan 250g", 120_000, 400);
       Product ebook = new DigitalProduct("EBK-003", "E-book Resep
Kue Nusantara", 50_000, "https://toko-umkm.example/unduh/ebk-003");
       List<Product> cart = List.of(coffee, rendang, ebook);
       System.out.println("== Katalog ==");
       for (Product product : cart) {
        System.out.println(product.getInfo());
    }
    CheckoutService checkout = new CheckoutService();
    ShippingService regular = new RegularShipping();
    ShippingService express = new ExpressShipping();
    System.out.println("\n== Ringkasan ==");
    System.out.println("Subtotal        : Rp" +
checkout.subtotal(cart));
    System.out.println("Berat kiriman   : " +
checkout.totalWeight(cart) + " g");
    System.out.println("Total Reguler   : Rp" + checkout.total(cart,
regular));
    System.out.println("Total Ekspres   : Rp" + checkout.total(cart,
express));
    }
}
```

Penjelasan: Menguji pembentukan keranjang berisi PhysicalProduct dan DigitalProduct, lalu menampilkan daftar katalog, subtotal harga, total berat barang fisik, serta kalkulasi estimasi ongkos kirim menggunakan kurir RegularShipping dan ExpressShipping.

output ShopApp setelah L2:

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal        : Rp250000
Berat kiriman   : 700 g
Total Reguler   : Rp260000
Total Ekspres   : Rp268000
```

SelfCheck L2.1–L2.4 lulus. L2.5 baru lulus setelah L3.

```
-- L2 - Interface & program to an interface
  [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
  [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
  [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
  [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan class konkret)
```

L3 Interface lanjutan

L3.a- Class Discountable

```java
package shop.discount;


public interface Discountable {
    int discountPercent(); // abstract: each class decides

    default long applyDiscount(long price) {
         return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
      int percent = discountPercent();
      if (percent < 0) {
        return 0;
      }
      return Math.min(percent, 100);
    }
}
```

Penjelasan: Interface Discountable menyediakan default method applyDiscount() untuk menghitung harga setelah potongan, static method

percentOf() untuk kalkulasi persentase, serta private method safePercent() untuk memvalidasi batasan persentase diskon $(0 - 100\%)$. Kelas implementasi cukup menyediakan persentase pada discountPercent().

L3.b- Class DigitalProduct ikut promo

```
public class DigitalProduct extends Product implements Discountable
{
```

```
    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }
    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
```

Penjelasan: Kelas DigitalProduct diperbarui dengan menambahkan implements Discountable. Dengan menetapkan konstanta DIGITAL_DISCOUNT_PERCENT = 10 dan meng-override getFinalPrice() untuk memanggil applyDiscount(), harga e-book otomatis terpotong 10% tanpa mengubah logika transaksi pada CheckoutService.

L3.c- Interface inheritance.

```
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);
}
```

```
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
```

```
            throw new IllegalArgumentException(
                "Months must be between 1 and " +
maxInstallmentMonths());
        }
        return (amount + months - 1L) / months;
    }
}
```

Penjelasan: Interface inheritance menggunakan kata kunci extends. RefundablePayment memperluas PaymentMethod dengan kemampuan refund(long amount), sedangkan InstallmentPayment menambahkan kontrak maxInstallmentMonths() serta default method monthlyInstallment() untuk menghitung nilai cicilan per bulan.

L3.d-

```
package shop.payment;

public class EWalletPayment implements RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be
blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be
negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }
```

```
    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Penjelasan: Diubah menjadi implements RefundablePayment dan mengimplementasikan method refund(amount) yang akan menambahkan kembali saldo balance jika pemrosesan refund valid.

```
package shop.payment;

public class CreditCardPayment implements RefundablePayment,
InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        if (cardHolder == null || cardHolder.isBlank()) {
            throw new IllegalArgumentException("Card holder must not
be blank");
        }
        if (creditLimit <= 0) {
            throw new IllegalArgumentException("Credit limit must be
greater than zero");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }
```

```
    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    public long getUsedCredit() {
        return usedCredit;
    }
}
```

Penjelasan: Mengimplementasikan interface RefundablePayment dan InstallmentPayment. Mengelola limit kredit creditLimit dan penggunaan kredit usedCredit untuk transaksi pay(), refund(), serta perhitungan cicilan hingga 12 bulan.

```
    public boolean refund(RefundablePayment payment, long amount) {
    return payment.refund(amount);
}
```

Method refund() pada CheckoutService: Menambahkan method refund yang secara khusus hanya menerima jenis pembayaran yang mendukung fitur refund

```
    // Poin 4 L3.d: Uji coba refund dengan BankTransferPayment
(dijadikan komentar)
    // Karena BankTransferPayment tidak mengimplementasikan
RefundablePayment,
    // compiler Java akan menolak baris di bawah ini sebelum program
berjalan.
    //checkout.refund(new BankTransferPayment("Bank Nagari"),
1_000);
```

Percobaan memanggil checkout.refund(new BankTransferPayment("Bank Nagari"), 1000) menghasilkan compile error karena BankTransferPayment tidak mengimplementasikan RefundablePayment. Baris ini kemudian dijadikan komentar.

L3.e- Lengkapi ShopApp dengan pembayaran

```
        EWalletPayment wallet = new EWalletPayment("Dompetku",
300_000);
        PaymentMethod transfer = new BankTransferPayment("Bank
Nagari");

        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() + " berhasil? " +
checkout.checkout(cart, regular, wallet));
        System.out.println("Sisa saldo: Rp" + wallet.getBalance());

        System.out.println(transfer.getName() + " berhasil? " +
checkout.checkout(cart, express, transfer));

        CreditCardPayment card =new CreditCardPayment("Siti Rahma",
1_000_000);
        System.out.println(card.getName() + " berhasil? " +
checkout.checkout(cart, express, card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart, express), 3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet berhasil? " +
checkout.refund(wallet, ebook.getFinalPrice()));
```

```
        System.out.println("Saldo setelah refund: Rp" +
wallet.getBalance());
```

Penjelasan: Melengkapi kelas ShopApp untuk menguji pemrosesan transaksi menggunakan EWalletPayment, BankTransferPayment, dan CreditCardPayment serta menguji pengembalian dana pada produk digital e-book.

Contoh output ShopApp setelah L3

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal        : Rp250000
Berat kiriman   : 700 g
Total Reguler   : Rp260000
Total Ekspres   : Rp268000

== Pembayaran ==
E-Wallet Dompetku berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000
```

SelfCheck L2.5 dan L3.1–L3.6 lulus

```
  [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000

-- L3 - Default, static, private method; interface inheritance
  [LULUS] L3.1 Discountable: applyDiscount default, percentOf static, ada private method
  [LULUS] L3.2 DigitalProduct implements Discountable; harga akhir 50000 -> 45000
  [LULUS] L3.3 RefundablePayment & InstallmentPayment adalah interface turunan PaymentMethod
  [LULUS] L3.4 CreditCardPayment implements RefundablePayment DAN InstallmentPayment
  [LULUS] L3.5 monthlyInstallment(100000, 3)=33334 dan 13 bulan ditolak
  [LULUS] L3.6 CheckoutService.refund(...) hanya menerima RefundablePayment
```

L4 final method dan final class

L4.a- Kunci rumus ongkir

```
    public final long calculateCost(int weightGrams) {
```

Penjelasan: Method calculateCost() pada ShippingService ditambahkan modifier final agar alur dasar pembulatan berat dan rumus ongkir tidak bisa diubah atau di override oleh kelas kurir turunan.

```
//package shop.shipping;

//public class FreeShipping extends ShippingService {
    //public FreeShipping() {
        //super("Gratis");
    //}

    //@Override
    //protected long ratePerKg() {
        //return 0;
    //}

    //@Override
    //public long calculateCost(int weightGrams) {
        //return 0;
    //}
//}

//pesan error muncul
//Cannot override the final method from ShippingService
```

Uji coba pembuatan kelas FreeShipping yang mencoba meng-override method ini dibuktikan menghasilkan error compiler: "Cannot override the final method from ShippingService".

L4.b- Address immutable

```
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank()) {
```

```
            throw new IllegalArgumentException("Street must not be
blank");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must not be
blank");
        }
        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal code must not
be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Kelas Address dirancang sebagai final class dengan semua field private final dan tanpa setter agar bersifat immutable. Modifier final pada kelas mencegah pihak luar membuat kelas turunan yang dapat merusak jaminan sifat immutable.

SelfCheck L4.1–L4.2 lulus.

```
-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter
```

L5 Nested class: static nested, inner, dan anonymous class

L5.a dan L5.b-Class Order

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress
== null) {
            throw new IllegalArgumentException("Order id and address
are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }

    public Address getShippingAddress() {
        return shippingAddress;
    }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
```

```
            if (product == null || quantity <= 0) {
                throw new IllegalArgumentException("Product is
required and quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() {
            return product;
        }

        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }

    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }

    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(Order.this.orderId).append("\n");
            for (Line line : lines) {
                sb.append("- ")
                  .append(line.getProduct().getName())
                  .append(" x")
                  .append(line.getQuantity())
                  .append(" = Rp")
                  .append(line.subtotal())
                  .append("\n");
            }
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke:
").append(Order.this.shippingAddress);
```

```
            return sb.toString();
        }
    }

    public Receipt createReceipt() {
        return new Receipt();
    }
}
```

1. **public static class Line:** Digunakan sebagai kelas pembantu internal untuk merepresentasikan baris pesanan yang tidak memerlukan akses ke atribut/objek luar Order.
2. **getItemsTotal():** Method untuk menghitung total keseluruhan subtotal dari seluruh Line barang di dalam pesanan.
3. **public class Receipt:** Merupakan kelas cetak struk yang terikat langsung dengan instance Order. Mampu mengakses data luar orderId dan shippingAddress secara langsung melalui sintaks Order.this.
4. **createReceipt():** Method untuk mengembalikan objek Receipt baru.

L5.c- Lengkapi ShopApp.

```
        Address address = new Address("Jl. Moh Hatta No. 10",
"Padang", "25121");
        Order order = new Order("ORD-0001", address);
        order.addLine(coffee, 2);
        order.addLine(ebook, 1);
        Order.Receipt receipt = order.new Receipt();
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;

            @Override
            public String getName() {
                return "Voucher Promo Hari Ini";
            }

            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
```

```
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + " untuk e-book
berhasil? "
            + checkout.checkout(List.of(ebook), regular, voucher));
```

Penjelasan: Menyempurnakan ShopApp dengan membuat objek Address, Order, memasukkan item barang, serta mencetak struk menggunakan inner class order.new Receipt(). Selain itu, membuat pembayaran promo sekali pakai yang mengimplementasikan interface PaymentMethod secara langsung tanpa membuat file kelas terpisah.

Contoh output ShopApp setelah L5

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal        : Rp250000
Berat kiriman   : 700 g
Total Reguler   : Rp260000
Total Ekspres   : Rp268000

== Pembayaran ==
E-Wallet Dompetku berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000

STRUK PESANAN ORD-0001
- Kopi Arabika Gayo 250g x2 = Rp170000
- E-book Resep Kue Nusantara x1 = Rp45000
Total barang: Rp215000
Kirim ke: Jl. Moh Hatta No. 10, Padang 25121

Voucher Promo Hari Ini untuk e-book berhasil? true
PS D:\praktikum6>
```

SelfCheck L5.1–L5.3 lulus. Target akhir: 23 / 23

```
-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
PS D:\praktikum6>
```

TUGAS RUMAH

E.0 Parameter pribadi berbasis NIM

```
Parameter tugas untuk NIM 2511532019
  CARD_ADMIN_PERCENT  = 1
  TRANSFER_ADMIN_FEE  = 4500
  SMS_MAX_LENGTH      = 119
```

Salin ketiga nilai yang dicetak ke src/shop/StudentInfo.java, lalu isi juga NIM dan NAME.

```java
package shop;

/**
 * Identity and personal parameters for the take-home assignment (Tugas).
 * 1. Write your NIM and name.
 * 2. Run: java -cp out tools.NimParams <your NIM>
 * 3. Copy the three printed values below.
 */
public final class StudentInfo {
    public static final String NIM = "2511532019";
    public static final String NAME = "Tiara Amalia Insani";

    public static final int CARD_ADMIN_PERCENT = 1;
    public static final long TRANSFER_ADMIN_FEE = 4500;
    public static final int SMS_MAX_LENGTH = 119;

    private StudentInfo() {
        // constants only; no objects
    }
}
```

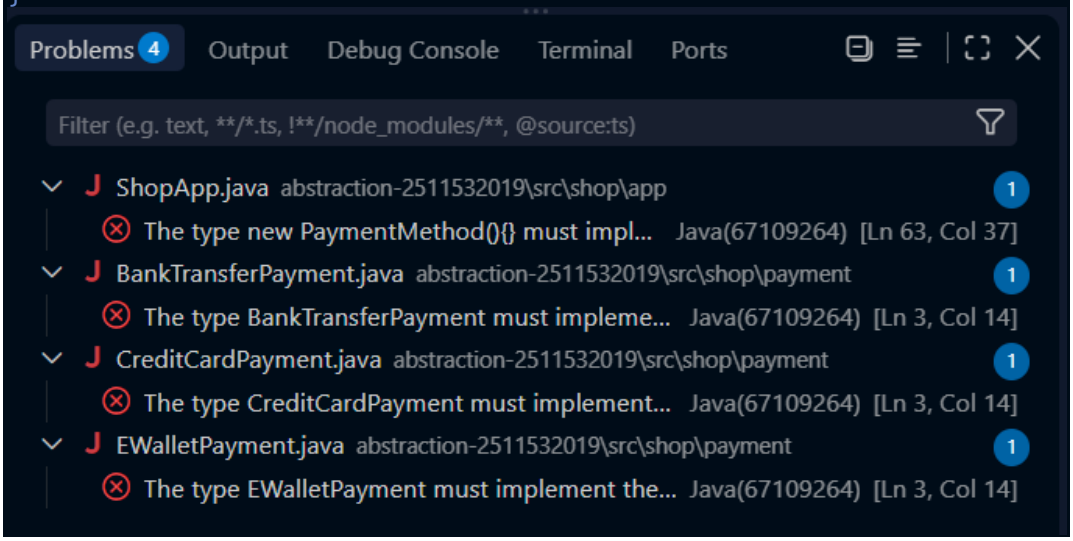
T1.a-Cara yang salah dulu

Tambahkan method abstract long adminFee(long amount); ke PaymentMethod, lalu kompilasi

```java
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();
```

```
    /** @return true if the payment succeeds */
    boolean pay(long amount);
    long adminFee(long amount);
}
```



ada berapa error : 4

class mana saja: ShopApp.java, BankTransferPayment.java, CreditCardPayment.java, EWalletPayment.java

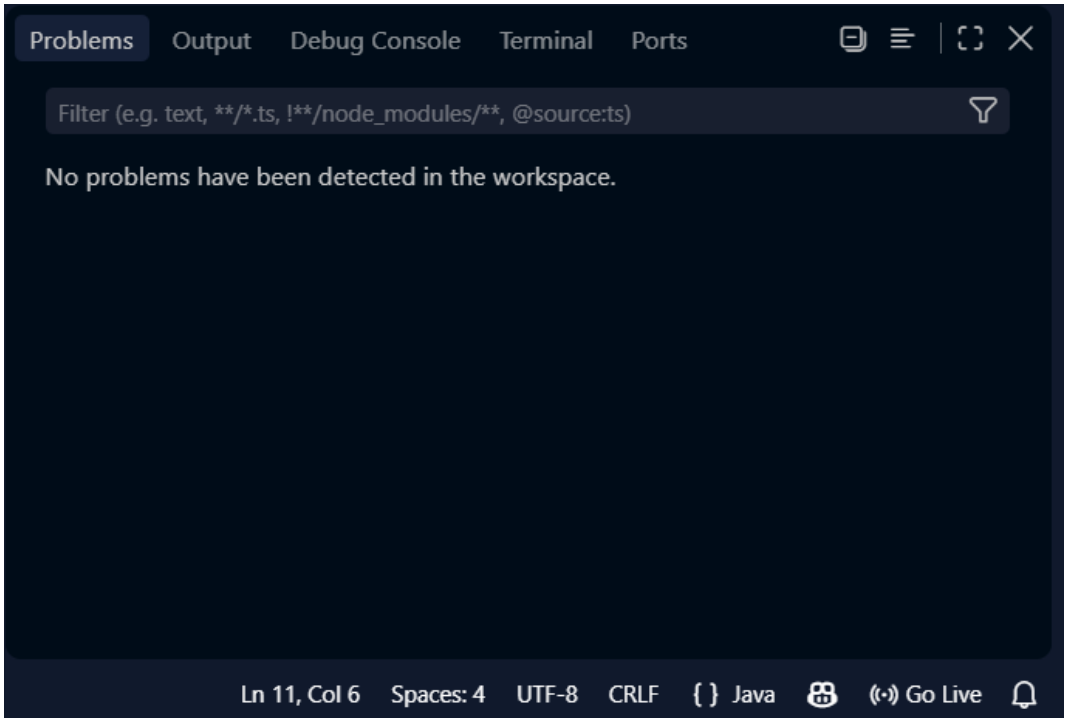
mengapa anonymous class voucher di ShopApp juga ikut rusak? Karena anonymous class pada ShopApp bertindak sebagai bentuk implementasi instan dari PaymentMethod, kelas anonim tersebut juga terkena aturan kontrak yang sama. Jika ia tidak meng-override method adminFee, compiler Java akan langsung menganggapnya tidak lengkap dan memunculkan compile error

T1.b-Default method

```
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);
    default long adminFee(long amount) {
        return 0;
    }
}
```



Error compiler tersebut hilang karena penambahan keyword default menyediakan implementasi bawaan pada interface, sehingga kelas-kelas implementor tidak lagi wajib meng-override method tersebut.

T1.c-Override sesuai kebutuhan.

Class CreditCardPayment

```
    @Override
    public long adminFee(long amount) {
        return (amount * StudentInfo.CARD_ADMIN_PERCENT) / 100;
    }
```

Class BankTransferPayment

```
    @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
```

T1.d-CheckoutService

```
    public boolean checkout(List<Product> items, ShippingService
shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
```

```
            return false;
        }
        long finalAmount = grandTotal(items, shipping, payment);
        return payment.pay(finalAmount);
    }
```

Penambahan method grandTotal pada CheckoutService berfungsi untuk menghitung total keseluruhan pembayaran yang menggabungkan subtotal keranjang, ongkos kirim, dan biaya admin dari metode pembayaran yang digunakan.

T1.e-Konflik diamond.

```
package shop.payment;

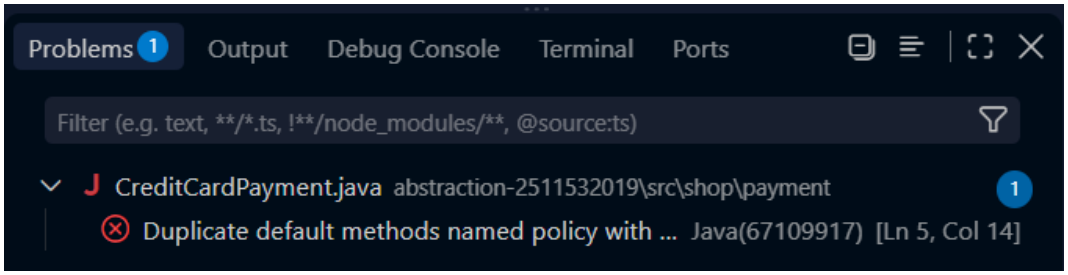
public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);

    default String policy() {
    return "Refund maksimal 7 hari";
    }
}
```

```
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                "Months must be between 1 and " +
maxInstallmentMonths());
        }
        return (amount + months - 1L) / months;
    }
    default String policy() {
    return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }
}
```



CreditCardPayment error karena kelas tersebut mengimplementasikan dua interface sekaligus (RefundablePayment dan InstallmentPayment), di mana kedua interface tersebut sama-sama memiliki default method bernama policy(). Ini memicu konflik pewarisan ganda sehingga compiler Java mengalami kebingungan dalam menentukan method policy() mana yang harus diwarisi, dan mewajibkan pengembang untuk meng-override-nya.

meng-override policy() di CreditCardPayment

```
    @Override
    public String policy() {
        return RefundablePayment.super.policy() + "; " +
InstallmentPayment.super.policy();
    }
```

1) Mengapa EWalletPayment tidak perlu meng-override policy()?
   ➔ Karena EWalletPayment hanya mengimplementasikan RefundablePayment (tidak mengimplementasikan InstallmentPayment), sehingga ia hanya mewarisi satu method policy() tunggal dari RefundablePayment tanpa mengalami konflik turunan ganda (tidak ada diamond problem).
2) Mengapa Java tidak memilih salah satu default secara otomatis?
   ➔ Java menghindari pemilihan otomatis untuk mencegah ambiguitas dan side-effect tak terduga pada logika bisnis, sehingga kompiler memaksa pengembang untuk menyelesaikan konflik secara eksplisit.

Checkpoint T1: TugasCheck P.1 dan T1.1–T1.4 lulus.

```
Problems  Output  Debug Console  Terminal  Ports

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp4500, 119 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

T2-Merancang sistem notifikasi pesanan

UML(PlantUML)

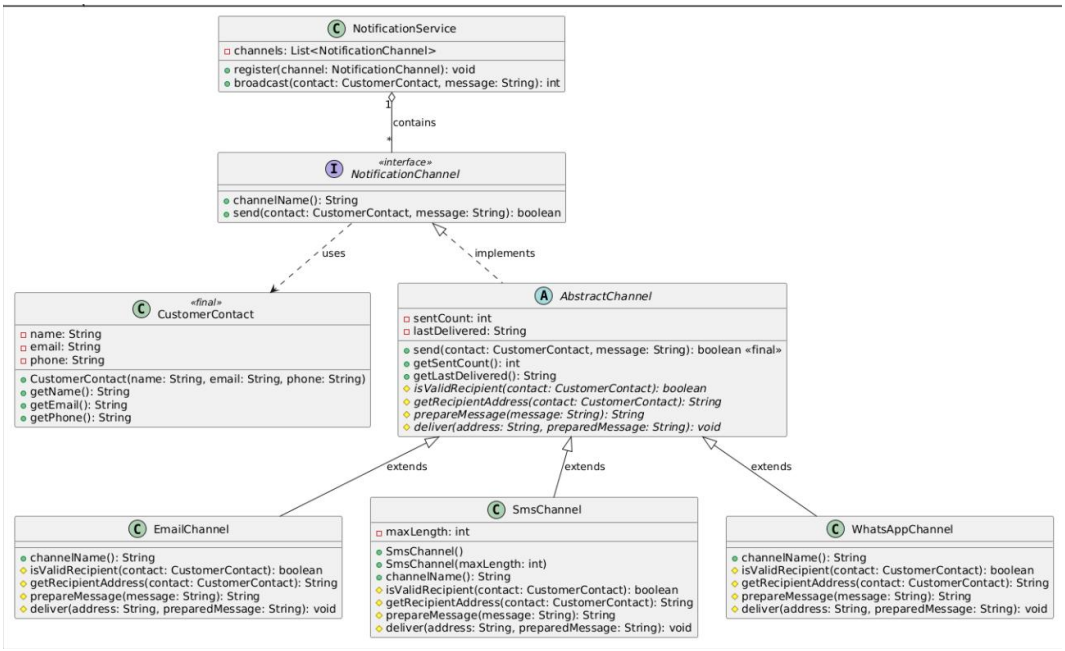


Diagram UML tersebut memodelkan struktur sistem notifikasi berbasis polimorfisme, yang terdiri dari interface NotificationChannel, kelas abstrak AbstractChannel, tiga kelas konkret kanal pengiriman, serta kelas pengelola NotificationService dan data kontak CustomerContact.

Implementasikan

Class CustomerContact

```
package shop.notification;


public final class CustomerContact {
```

```
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email, String phone)
{
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be
blank");
        }
        this.name = name;
        this.email = (email == null) ? "" : email;
        this.phone = (phone == null) ? "" : phone;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPhone() {
        return phone;
    }
}
```

Kelas immutable yang merepresentasikan data informasi kontak pelanggan (nama, email, dan nomor telepon) untuk keperluan pengiriman pesan.

interface NotificationChannel

```
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String message);
}
```

Interface utama yang mendefinisikan kontrak dasar bagi seluruh kanal pengiriman notifikasi melalui method channelName() dan send().

Class AbstractChannel

```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel
{
    private int sentCount = 0;
    private String lastDelivered = "";

    protected abstract boolean isValidRecipient(CustomerContact
contact);
    protected abstract String getRecipientAddress(CustomerContact
contact);
    protected abstract String prepareMessage(String message);
    protected abstract void deliver(String address, String
preparedMessage);

    @Override
    public final boolean send(CustomerContact contact, String
message) {
        if (contact == null || message == null || message.isBlank())
{
            return false;
        }
        if (!isValidRecipient(contact)) {
            return false;
        }
        String address = getRecipientAddress(contact);
        if (address == null || address.isBlank()) {
            return false;
        }
        String prepared = prepareMessage(message);
        deliver(address, prepared);

        sentCount++;
        lastDelivered = prepared;
        return true;
    }

    public int getSentCount() {
        return sentCount;
    }

    public String getLastDelivered() {
        return lastDelivered;
    }
}
```

Kelas abstrak yang mengimplementasikan interface NotificationChannel dengan menerapkan pola Template Method untuk mengunci alur pengiriman pesan utama menggunakan method final.

Class EmailChannel

```
package shop.notification;

public class EmailChannel extends AbstractChannel {
    @Override
    public String channelName() {
        return "EMAIL";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact contact) {
        String email = contact.getEmail();
        return email != null && email.contains("@") &&
!email.isBlank();
    }

    @Override
    protected String getRecipientAddress(CustomerContact contact) {
        return contact.getEmail();
    }

    @Override
    protected String prepareMessage(String message) {
        return message;
    }

    @Override
    protected void deliver(String address, String preparedMessage) {
        System.out.println("[EMAIL to " + address + "] " +
preparedMessage);
    }
}
```

Kelas turunan AbstractChannel yang menangani spesifikasi pengiriman notifikasi khusus melalui kanal surat elektronik (email).

Class SmsChannel

```
package shop.notification;
```

```java
import shop.StudentInfo;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel() {
        this.maxLength = StudentInfo.SMS_MAX_LENGTH;
    }

    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() {
        return "SMS";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        return phone != null && phone.startsWith("08");
    }

    @Override
    protected String getRecipientAddress(CustomerContact contact) {
        return contact.getPhone();
    }

    @Override
    protected String prepareMessage(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) + "...";
        }
        return message;
    }

    @Override
    protected void deliver(String address, String preparedMessage) {
        System.out.println("[SMS to " + address + "] " +
preparedMessage);
    }
}
```

Kelas turunan AbstractChannel yang mengatur logika pengiriman pesan singkat (SMS) lengkap dengan fitur pemotongan otomatis teks yang melebihi batas karakter.

Class WhatsAppChannel

```
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {
    @Override
    public String channelName() {
        return "WA";
    }

    @Override
    protected boolean isValidRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        if (phone == null) return false;
        return (phone.startsWith("08") || phone.startsWith("+62"))
&& phone.length() >= 10;
    }

    @Override
    protected String getRecipientAddress(CustomerContact contact) {
        return contact.getPhone();
    }

    @Override
    protected String prepareMessage(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected void deliver(String address, String preparedMessage) {
        System.out.println("[WA to " + address + "] " +
preparedMessage);
    }
}
```

Kelas turunan AbstractChannel yang mengimplementasikan aturan validasi nomor handphone dan format pesan khusus untuk pengiriman via WhatsApp.

Class NotificationService

```java
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
    private final List<NotificationChannel> channels = new
ArrayList<>();

    public void register(NotificationChannel channel) {
        if (channel != null) {
            channels.add(channel);
        }
    }

    public int broadcast(CustomerContact contact, String message) {
        int successCount = 0;
        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                successCount++;
            }
        }
        return successCount;
    }
}
```

Kelas pengelola utama yang bertindak sebagai wadah (registry) penyimpanan daftar kanal dan melakukan broadcast pesan secara polimorfik ke seluruh kanal yang terdaftar.

Demo

Class TugasApp

```java
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.ShippingService;
```

```java
import shop.notification.*;
import shop.StudentInfo;
import shop.payment.RefundablePayment;

public class TugasApp {
    public static void main(String[] args) {
        // Produk keranjang contoh
        Product coffee = new PhysicalProduct("KOPI-001", "Kopi
Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-002", "Rendang
Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003", "E-book Resep
Kue Nusantara", 50_000, "https://toko-umkm.example/unduh/ebk-003");
        List<Product> cart = List.of(coffee, rendang, ebook);

        CheckoutService checkout = new CheckoutService();
        ShippingService express = new ExpressShipping();

        PaymentMethod wallet = new EWalletPayment("Dompetku",
300_000);
        PaymentMethod transfer = new BankTransferPayment("Bank
Nagari");
        CreditCardPayment card = new CreditCardPayment("Siti Rahma",
1_000_000);

        System.out.println("Tugas oleh " + StudentInfo.NAME + " (" +
StudentInfo.NIM + ")");
        System.out.println("== T1: Total dengan biaya admin
(Ekspres) ==");
        System.out.println(wallet.getName() + " Rp" +
checkout.grandTotal(cart, express, wallet) + " (admin Rp" +
wallet.adminFee(checkout.total(cart, express)) + ")");
        System.out.println(transfer.getName() + " Rp" +
checkout.grandTotal(cart, express, transfer) + " (admin Rp" +
transfer.adminFee(checkout.total(cart, express)) + ")");
        System.out.println(card.getName() + ": Rp" +
checkout.grandTotal(cart, express, card) + " (admin Rp" +
card.adminFee(checkout.total(cart, express)) + ")");

        System.out.println("Kebijakan kartu kredit : " +
card.policy());
        System.out.println("Kebijakan e-wallet     : " +
((RefundablePayment)wallet).policy());

        System.out.println("\n== T2: Notifikasi status pesanan ==");
        CustomerContact siti = new CustomerContact("Siti Rahma",
"siti@example.com", "081234567890");
```

```
        CustomerContact budi = new CustomerContact("Budi", null,
"+6281298765432");

        String longMessage = "Pesanan ORD-0001 sudah dikirim dengan
kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam.
Terima kasih telah berbelanja produk UMKM Sumatera Barat di toko
kami!";

        NotificationService service = new NotificationService();
        service.register(new EmailChannel());
        service.register(new SmsChannel());
        service.register(new WhatsAppChannel());

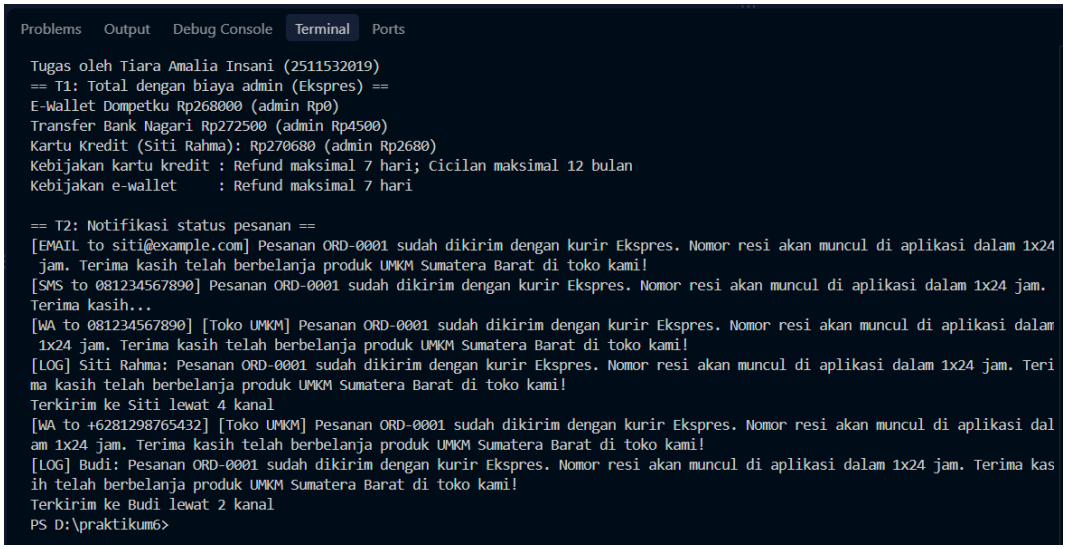
        // Anonymous class untuk log konsol
        service.register(new NotificationChannel() {
            @Override
            public String channelName() {
                return "LOG";
            }
            @Override
            public boolean send(CustomerContact contact, String
message) {
                System.out.println("[LOG] " + contact.getName() + ":
" + message);
                return true;
            }
        });

        int sentSiti = service.broadcast(siti, longMessage);
        System.out.println("Terkirim ke Siti lewat " + sentSiti + "
kanal");

        int sentBudi = service.broadcast(budi, longMessage);
        System.out.println("Terkirim ke Budi lewat " + sentBudi + "
kanal");
    }
}
```

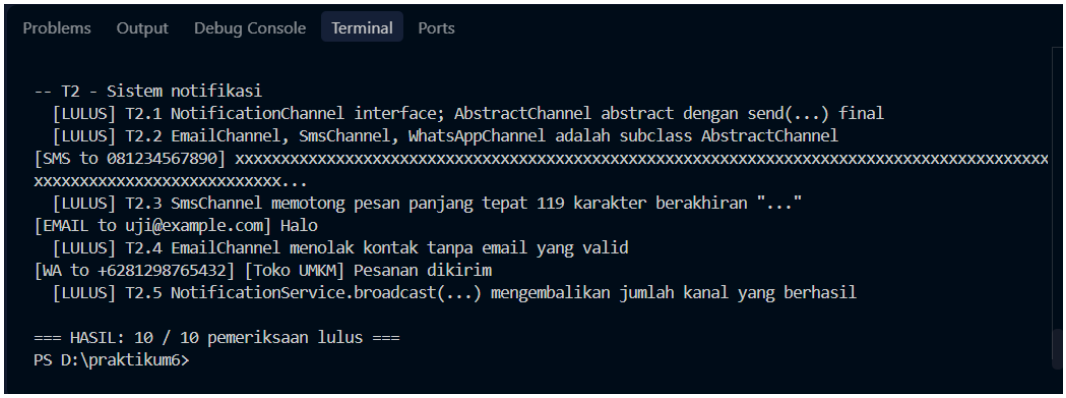
Kelas aplikasi utama untuk menjalankan dan menguji keseluruhan skenario tugas rumah, mulai dari kalkulasi biaya admin pembayaran hingga sistem broadcast notifikasi multi-kanal

Output TugasApp



1. Mengapa ada interface dan abstract class, bukan salah satunya saja?
   Interface digunakan untuk mendefinisikan kontrak kemampuan murni yang wajib dipatuhi oleh kelas tanpa terikat hierarki pewarisan. Sementara itu, abstract class digunakan untuk menyediakan kerangka kode bersama agar tidak terjadi pengulangan kode di antara kelas-kelas.
2. Apa akibatnya jika send(...) tidak dibuat final?
   kelas turunan dapat dengan bebas mengubah alur logika utama pengiriman pesan seperti validasi penerima atau pencatatan riwayat. Akibatnya, konsistensi prosedur pengiriman pesan yang sudah distandarisasi di dalam template method bisa rusak
3. Mengapa NotificationService menyimpan List, bukan List?
   Karena penggunaan tipe data polimorfik berbasis interface memungkinkan sistem untuk menampung dan mengelola berbagai jenis kanal pengiriman yang berbeda seperti Email, SMS, WhatsApp, hingga Log kustom ke dalam satu wadah daftar yang sama.

TugasCheck T2.1–T2.5 lulus.


Problems
Output
Debug Console
Terminal
Ports
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

		diubah dari luar tanpa enkapsulasi).	program berjalan.	bersifat immutable.
3.	Express Shipping / calculateCost()	Desain Abstract Class / Method Kalkulasi Ongkir yang Cacat (mengembali kan nilai 0 pada kondisi tertentu).	Ongkir bernilai 0 untuk pengiriman ekspres yang seharusnya berbayar, merugikan pendapatan bisnis toko online.	Perbaiki logika perhitungan di dalam kelas konkret pengiriman agar mengembali kan tarif yang benar sesuai berat/jarak.
4	Interface Umum / Default Method	Penyalahgunaan Default Method atau Kontrak Kosong yang tidak di- override dengan benar oleh kelas turunan.	Menyebabkan perilaku fungsi bernilai salah seperti validasi pembayaran true | Voucher yang meloloskan transaksi secara tidak semestinya	Pastikan setiap kelas turunan wajib meng-override method abstrak utama daripada mengandalkan perilaku default yang kosong.
5	Nested Class / Inner Class	Penggunaan Inner Class Non-Static yang Terikat Erat secara tidak tepat pada logika transaksi bisnis utama.	Menyebabkan tight coupling yang menyulitkan pengujian unit secara terisolasi dan meningkatkan risiko memory leak.	Ubah menjadi static nested class atau pisahkan menjadi kelas top-level biasa yang independen.

Class BadShopRefactored

```java
package review.fixed;

import java.util.ArrayList;
import java.util.List;

interface ShippingService {
    String getName();
    long calculateCost(int weightGrams);
}

class RegularShipping implements ShippingService {
    @Override
    public String getName() {
        return "REGULER";
    }

    @Override
    public long calculateCost(int weightGrams) {
        int weightKg = (int) Math.ceil(weightGrams / 1000.0);
        return weightKg * 10000L;
    }
}

class ExpressShipping implements ShippingService {
    @Override
    public String getName() {
        return "EXPRES";
    }

    @Override
    public long calculateCost(int weightGrams) {
        int weightKg = (int) Math.ceil(weightGrams / 1000.0);
        return weightKg * 20000L;
    }
}

interface Shippable {
    int getWeightGrams();
}

abstract class Product {
    private final String code;
    private final String name;
    private final long price;
```

```
    public Product(String code, String name, long price) {
        this.code = code;
        this.name = name;
        this.price = price;
    }

    public String getCode() { return code; }
    public String getName() { return name; }
    public long getPrice() { return price; }
}

class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String code, String name, long price, int
weightGrams) {
        super(code, name, price);
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }
}

class GiftVoucher extends Product {
    public GiftVoucher(String code, String name, long price) {
        super(code, name, price);
    }
}

class Address {
    private final String street;
    private final String city;

    public Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    public String getStreet() { return street; }
    public String getCity() { return city; }

    @Override
    public String toString() {
        return street + ", " + city;
```

```
    }
}

class Order {
    private final String orderId;
    private final Address address;
    private final List<Product> items;

    public Order(String orderId, Address address, List<Product>
items) {
        this.orderId = orderId;
        // Defensive copy / Immutable reference
        this.address = new Address(address.getStreet(),
address.getCity());
        this.items = new ArrayList<>(items);
    }

    public String getOrderId() { return orderId; }
    public Address getAddress() { return address; }
    public List<Product> getItems() { return items; }
}

interface PaymentMethod {
    String getName();
    boolean pay(long amount);
}

class VoucherPayment implements PaymentMethod {
    private double balance;

    public VoucherPayment(double balance) {
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "Voucher";
    }

    @Override
    public boolean pay(long amount) {
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
```

```
    public double getBalance() {
        return balance;
    }
}

public class BadShopRefactored {
    public static void main(String[] args) {
        ShippingService express = new ExpressShipping();
        int cartWeight = 2000; // 2 kg
        long expressCost = express.calculateCost(cartWeight);

        System.out.println("Total cart: 170000");
        System.out.println("Ongkir " + express.getName() + " 2 kg: "
+ expressCost);

        Address addr1 = new Address("Jl. Sudirman 10",
"Bukittinggi");
        Order ord1 = new Order("ORD-1", addr1, List.of());

        Address addr2 = new Address("Jl. M. Hatta 15", "Padang");
        Order ord2 = new Order("ORD-2", addr2, List.of());

        System.out.println(ord1.getOrderId() + " -> " +
ord1.getAddress());
        System.out.println(ord2.getOrderId() + " -> " +
ord2.getAddress());

        Product flashSaleItem = new PhysicalProduct("FLASH-1",
"Sepatu Lokal", 70000, 500);
        Product voucherItem = new GiftVoucher("VOUCH-100", "Voucher
Belanja 100k", 100000);

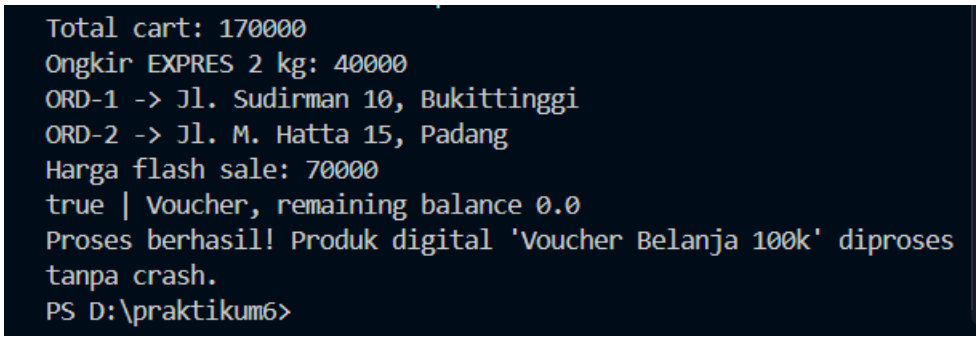
        System.out.println("Harga flash sale: " +
flashSaleItem.getPrice());

        VoucherPayment voucherPayment = new VoucherPayment(70000.0);
        boolean payResult =
voucherPayment.pay(flashSaleItem.getPrice());

        System.out.println(payResult + " | " +
voucherPayment.getName() + ", remaining balance " +
voucherPayment.getBalance());
        System.out.println("Proses berhasil! Produk digital '" +
voucherItem.getName() + "' diproses tanpa crash.");
    }
}
```

merangkum hasil perbaikan dari desain kode awal melalui penerapan pemisahan interface, seperti memisahkan Shippable untuk produk fisik dan membiarkan GiftVoucher mewarisi Product secara independen. Selain itu, kelas Order menerapkan defensive copying pada atribut alamat dan daftar item guna mencegah mutable shared state, serta menggunakan struktur polimorfik pada layanan pengiriman dan pembayaran agar program berjalan aman tanpa risiko runtime crash.

Output BadShopRefactored



T4- Refleksi: kapan memakai apa

Tabel 1- Abstract class vs interface

Pertanyaan desain	Abstract class atau interface?	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	Abstract class	Hanya abstract class yang bisa memiliki variabel instance dan constructor untuk menginisialisasi data bersama, sedangkan interface hanya berisi public static final.	AbstractChannel memiliki variabel sentCount dan lastDelivered yang diwariskan ke kanal pengiriman.
Satu class perlu beberapa	Interface	Java tidak mendukung pewarisan ganda untuk class, tapi	CreditCardPayment meng implementasikan

kemampuan sekaligus		memperbolehkan sebuah kelas mengimplementasikan banyak interface sekaligus untuk mendapatkan banyak kemampuan.	RefundablePayment dan InstallmentPayment sekaligus.
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Memungkinkan kita menulis method konkrit yang diberi keyword final untuk mengunci alur utama agar tidak dirusak oleh subclass.	Method send(...) pada AbstractChannel yang dibuat final agar alur validasi dan pengirimannya konsisten.
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Menggunakan fitur default method di dalam interface memungkinkan penambahan method baru tanpa merusak kelas-kelas turunan yang sudah ada	Penambahan default method policy() atau adminFee() pada interface PaymentMethod.

Tabel 2- Jenis nested class

Kondisi	Static nested, inner, atau anonymous	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	Static nested	Bersifat mandiri dan tidak memerlukan referensi ke instance dari	Helper class internal untuk konfigurasi atau struktur data pembantu statis.

		outer class, sehingga lebih efisien.	
Objek harus membaca field objek luarnya	Inner (Non-static nested)	Memiliki akses langsung ke seluruh variabel dan method milik outer class instance.	Kelas internal yang membutuhkan akses langsung ke atribut instansiasi dari kelas utamanya.
Implementasi sekali pakai, tidak perlu nama	Anonymous	Sangat praktis untuk membuat instansiasi langsung dari suatu interface atau class secara instan di tempat tanpa harus mendeklarasikan nama kelas baru secara terpisah.	Pembuatan kanal log kustom dengan new NotificationChannel() { ... } di dalam ShopApp